

MongoDB

* MongoDB is a popular, one source, document-oriented NoSQL - (NoSQL) → Not only SQL refers to a category of non-relational databases that store data in flexible formats (like documents, key-value pairs, graphs, or wide columns) instead of traditional tables.

- database that stores data in flexible, JSON-like documents (BSON) instead of traditional tables and rows.

* In short you can say you it does not need tables like SQL it is in JSON format.

* To start mongoDB go to command prompt and type
-- mongosh

It ~~will~~ connect command prompt to MongoDB compass

* To clear the command prompt page
-- ctrl + L

* To see the database
-- show dbs

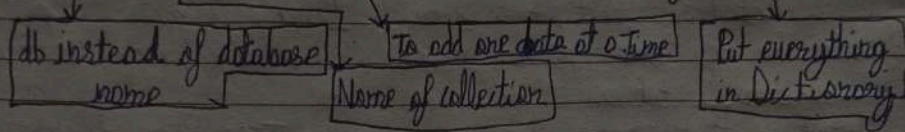
* To use and creating database ~~if~~ if not exist.
-- use database name

* To make like a data collection in database to like separate things from each other (collection in NoSQL - table in SQL)
-- db.createCollection("Name of collection")

* To see the collection from database
-- show collection

* To insert data in collection if you have not made the collection it will be made

```
-- db.students.insertOne({name: "Ram", age: 12})
```



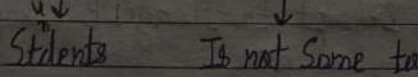
* To see the data you insert in the collection first 20

```
-- db.students.find()
```

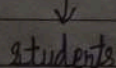
VERY IMPORTANT

It is a case sensitive language

```
-- db.Students.insertOne({name: "Ram", age: 12})
```



```
-- db.students.insertOne({name: "Mayank", age: 21})
```



* In Output you will get id it is a unique identifier for each document of a collection like Ram is a document and Mayank is different document both will have unique identifier.

* Nested Document

To add the data in the document if not exist it will be created if exist it will be updated only work on one document at a time

```
-- db.students.updateOne({name: "Ram", $set: {idcard: {hashcard: "falter", hashcard: "falter"}}
```

* To see data

```
-- db.Studen
```

VERY IMPORTANT
You can do
there is a
not go on

* To update

```
-- db.studen
```

and you

* To see data

```
-- db.stu
```

* CRUD

①. find
②. findOne
③. findMany
by u

collection

* To see data by name or any thing which is in document
 -- db.students.findOne({name: "Ram"})

VERY IMPORTANT

anything
 library

You can add as many nested document as you want to add but there is one thing you should know that the document size must not go over 16 mb

* To update all document at one time.

-- db.students.updateMany({}, {\$set: {hobbies: ["Anime", "Cooking"]}})

↓
 Empty because we
 want all to be updated

↓
 In list because
 you can put
 all in one list

and you can use " or ' both are same.

* To see data by Nested Documents

-- db.students.find({idCards.hasIdCard: true})

Why - because we set hasIdCard inside idCards

* CRUD Operations

for
 document

Create	⇒ C	R ⇒ Read
Update	⇒ U	D ⇒ Delete

* ①. find() give you first 20 document in a list

②. findOne() if it doesn't find something it will return null but

find() will give you a empty list it will give a cursor
 by cursor you can use function on it like .count(), limit(2)

ated
 time
 card: false

* To see the document less than
 -- db.students.find({age: {\$lt: 12}})

IMPORTANT

\$ = Reserved keyword only used in MongoDB

* To see the document less than or equal to
 -- db.students.find({age: {\$lte: 12}})

* To see the greater than the document
 -- db.students.find({age: {\$gt: 12}})

* To see the greater than or equal to the document
 -- db.students.find({age: {\$gte: 12}})

* To see the document between less than or greater than
 -- db.students.find({age: {\$gt: 5, \$lt: 12}})

* To see All the data from the collection
 -- db.students.find().toArray()

* While finding less than big then we can use other function to because it is using find()

* To insert the data in document

-- db.students.insert({name: "Ran", age: 12})

It will insert the data in document but insert is old way to add data but it is work but use insertOne or insertMany

* To insert one data at a time

-- db.students.insertOne({name: "Ran", age: 12})

* To insert Many data at a time
 -- db.students.insertMany([{\$name: "Vijay", age: 23}, {\$name: "Ran", age: 34}])
 while using insertMany put them in a list.

* To update the document of someone based on there data.
 -- db.students.updateOne({name: "Vijay"}, {\$set: {age: 15}})

To find what you want to update.

To set this New thing in place of old one

New thing

* To update all the data at a time
 -- db.students.updateMany({age: 12}, {\$set: {age: 13}})

* -- db.students.updateMany({age: 12}, {\$set: {isEligible: false}})
 you can use it to add new columns by updateMany and can use \$gt or \$lt

* To delete a One document from the collection.
 -- db.students.deleteOne({age: 13})

* To delete more than one document from the collection.
 -- db.students.deleteMany({age: 13})

* To delete All document from the collection.
 -- db.students.deleteMany({})

Empty

* Data types in MongoDB

- ① Text = Strings
- ② Boolean = ~~bit~~ True or False
- ③ Number = Integer = 32 bit
 NumberLong = 64 bit
 NumberDecimal = in Decimal
- ④ ObjectID = ID given to every document
- ⑤ ISO Date = Date
- ⑥ Timestamp = Millisecond
- ⑦ Array = To store ordered list of items within a single document.
- ⑧ Emb. document = Nested document

* ISO Date is used like
 foundedOn: new Date()

* Timestamp is used like
 foundedOnTimestamp: new Timestamp()

* To check the data type
 -- typeof db.companyData.findOne().name

↓
 To find data type then space

↓
 Collection Name

↓
 What you want to find
 data type of a column from Collection

* To Drop Database
 -- use mydb
 -- db.dropDatabase()



★ To Drop collections

-- db.products.new.drop()

★ You can give id also in a document

-- db.books.insertMany([{"id": "A", "name": "A", "price": 3}, {"id": "B", "name": "B", "price": 4}])

Some id can not be add now it will show error

-- db.books.insertMany([{"id": "C", "name": "C", "price": 5}, {"id": "A", "name": "A", "price": 6}, {"id": "D", "name": "D", "price": 7}])

Now it will show error but if you see the collections it will have A, B, C in it but not D because MongoDB got error after C so it stopped after C

-- db.books.insertMany([{"id": "D", "name": "D", "price": 8}, {"id": "A", "name": "A", "price": 9}, {"id": "E", "name": "E", "price": 10}])

Now it will show error but it will work after the error. Now we will have A, B, C, D, E

★ Validation ⇒ Making Rules

-- db.createCollection("nonfiction", {

validation: {

required: true, // Is a Schema

properties: {

name: {

mandatory: true

In properties you write about the requirements which we have mentioned what are the types and description if they are not given

type: "String",

description: "must be a string"

price: {

type: "number",

It is used to modify the collection

You can use in Val

description: "must be a number"

ValidationAction: "error"

By adding Error/warn in ValidationAction
error gives Error by running the code and Warn gives warning by running the code

By this you are giving rules to the collection.

* To alter or change the collection which was Validation

-- db.runCommand({

collMod: "nonfiction",

collMod telling the name of collection nonfiction

validator: {

\$JsonSchema: {

required: ["Name", "price", "author"],

properties: {

name: {

bsonType: "string",

description: "must be a string"

author: {

bsonType: "array",

description: "must be an array",

items: {

bsonType: "object",

required: ["name", "email"],

properties: {

You can use Nested Document in Validation.

We can use properties inside the properties.

name: {
 bsonType: "string"
 },
 email: {
 bsonType: "string"
 },
 }

ValidationAction: "warn"

will inserting in nested Document you have to put them in a list

* { w: <value>, J: <boolean>, wtimeout: <number> }
 = conseton

w

J

wtimeout

It give the acknowl-
 -gment it is like
 confirmation you get
 from mongo for the
 code you write is it
 done or not if it is
 stoped, code will not
 lost and it defaultly
 open in MongoDB

It is used to make a
 copy in a journal and
 in some data base like
 now it will save data in
 this place and it will
 know if code stop because
 of any thing it can be
 resume the code from
 where it stops it is bit
 slow it is off
 defaultly

You can give a timeout to
 code it is like if it take
 more then this time stop
 If done on time it will
 run if not on time it will
 not run. it is also off
 defaultly you have to set
 it up.

-- db: back
 { w: 1

* Atomicity
 Atomicity
 ① Fork
 ② Filt
 Atomicity
 are comp

* If You
 ① firstly
 down
 Men
 they

-- mongo
 studen
 - d =>
 - c =>
 -- Thanks

-- drop

① \$eq
 * \$eq
 ② \$lt
 ③ \$lte
 ④ \$in

↑ Comp

Types: "string"

type: "string"

-- db.books.insertOne({name: "B", price: 2}, {writeConcern: {w: 0, j: true, timeout: 1}})

* Atomicity in MongoDB

Atomicity: A database operation is all-or-nothing.

- ① Each operation on a single document is atomic.
- ② Either the entire update happens, or nothing happens.

Atomicity in MongoDB ensures that operations on a single document are completed fully or not executed at all.

* If you want to import a json file use data.

- ① Firstly Download MongoDB command line database Tools Download.

Then to much work go see the video video to much work may go to cmd and write

-- mongoimport "path of your json file" -d college -c students --jsonArray --drop

-d => Database if not made it will be made

-c => Collection if not made it will be made

--jsonArray => If you have not written this it will put all in one document but you have written this it will be in different document.

--drop => If exist it will be drop.

* ① \$eq = equal ② \$ne = not equal

③ \$lt = less than ④ \$gt = greater than

⑤ \$lte = less than or equal to ⑥ \$gte = greater than or equal to

⑦ \$in = in it ⑧ \$nin = not in it

↑ Comparison operators ↑

-- db.students.find({ age: { \$eq: 5 } })
 some till 6th operation
 -- db.students.find({ age: { \$in: [4, 5, 6] } })

★ To find Nested document

-- db.students.find({ "identity.hasAddress": true })

Address was in identity

★ Logical operators

- ① \$and → Both should be satisfy
 - ② \$or → satisfy any one of the argument
 - ③ \$nor → don't satisfy any argument
 - ④ \$not → some function like \$ne it will not show it and it is used with \$and
- db.students.find({ \$or: [{ age: { \$gt: 10 } }, { age: { \$gt: 12 } }] })

→ ~~not~~ some for 2 and 3

-- db.students.find({ \$and: [{ age: { \$lt: 11 } }, { age: { \$not: { \$not: { \$gt: 5 } } } }] })

★ Element Query Operators

- ① \$exists = finds you if it exist in document
 - ② \$type = find in document data type used with exist
- db.students.find({ hasMachook: { \$exists: true } })
 -- db.students.find({ hasMachook: { \$exists: true, \$type: "bool" } })
 you can see all data type in mongo documentation.

★ Evaluation Operators

- ① \$expr → Allow us to write a query in expr
- db.collection.find({ \$expr: { \$gt: ["\$field1", "\$field2"] } })

[337)

end": true})

if it and it is used with \$and
33, age: \$gte: 12

age: \$not: \$

ed with exist

e 33)

\$type: "bool" 33)

1, "field2" 7

This will find all documents in the "collection" where the value of "field1" is greater than the value of "field2"

② \$transSchema $\Rightarrow$ we used it in validator

③ \$regex $\Rightarrow$ To find something by telling what it start from
-- db.students.find({name: {\$regex: /^A/}})
Now it will show name starting with A

④ \$text $\Rightarrow$ It is like firstly ~~name~~ you change the index then you can search from it.

-- db.students.createIndex({bio: "text"})

you change the index to text

-- db.students.find({\$text: {\$search: "youtube"}})

⑤ \$mod $\Rightarrow$ it is using maths

-- db.students.find({age: {\$mod: [2, 0]}})

2 = divide by

0 = remainder

* \$size - tells the size

* \$elemMatch $\Rightarrow$ we use it in find to use it like \$and

* .sort() $\Rightarrow$ used after find() in sort write a num and if in it we put 1 or -1. 1 means Ascending -1 means descending

* To see all data in one time

in test -- ~~for~~ forEach(x $\Rightarrow$ printjson(x))

Mongo is written in Java script in backend so this is JavaScript code

★ .skip(20)

It is used in sort or limit and 20 tells no. of document of skip.

★ To increase or decrease int. values of document
 -- db.students.updateMany({ }, { \$inc: { age: 1 } })

↓
any filter

★ Use of \$ min or \$ max min means small than Max means big than

-- db.students.updateOne({ name: "Mayank" }, { \$min: { age: 23 } })

★ To multiply any int in the document

-- db.students.updateOne({ name: "Sita" }, { \$mul: { age: 23 } })

★ To delete one column from document

-- db.students.updateOne({ name: "Sita" }, { \$unset: { age: 1 } })

↓
(any value just to fill it)

★ To add column to document

-- db.students.updateOne({ name: "Sita" }, { \$set: { age: 23 } })

★ To rename column name

-- db.students.updateMany({ }, { \$rename: { age: "StudentAge" } })

↓
New name

★ To find out
 -- db.students.aggregate(\$project)

★ We use \$ project to
 -- db.students.aggregate(\$project)

★ To update
 -- db.students.updateOne(\$set: { age: 1 })

① \$ => it is used to set a value

② \$unset => it is used to delete a column

③ \$inc => it is used to increment a value

④ \$mul => it is used to multiply a value

⑤ \$div => it is used to divide a value

⑥ \$max => it is used to set a value to the maximum of the current value and the new value

⑦ \$min => it is used to set a value to the minimum of the current value and the new value

⑧ \$addToSet => it is used to add a value to an array

⑨ \$push => it is used to push a value to the end of an array

★ To add a new document to a collection

-- db.students.insertOne({ name: "Sita", age: 23 })

of document of.

document
: 1333)

then Max means

\$ min: { age: 233 }

mul: { age: 233 }

t: { age: 2333 }

↓
due just to fill

x: 2333)

lentAge 333)

ame

★ To find someone if not found insert it
-- db.students.updateOne({ name: "Ravi", \$set: { age: 10033 }, \$upsert: true })

★ We use \$elemMatch for to see the matching value in nested document.
-- db.products.find({ products: { \$elemMatch: { quantity: { \$gt: 100 }, name: "apple" } } })

★ To update something in nested nested document all in
-- db.students.updateMany({ experience: { \$elemMatch: { duration: { \$lte: 13333 }, \$set: { experience: { neglect: true } } } } })

①. \$. ⇒ it will update is it find but only first one given it has more than one in a nested document

②. neglect ⇒ is the name of the column

③. \$[] ⇒ it will update all in the document if it satisfy the argument or not satisfy all

★ To update all but it is important it to satisfy the argument.

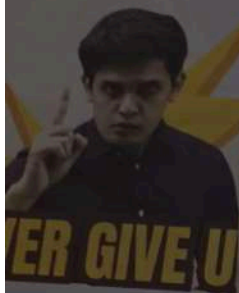
-- db.students.updateMany({ experience: { \$elemMatch: { duration: { \$lte: 13333 }, \$set: { experience: { \$[A].neglect: true } } }, \$arrayFilters: [{ "A.duration": { \$lte: 13333 } }] })

① \$[A] ⇒ "A" can be any thing it is just to name a function which will all neglect: true

② arrayFilters ⇒ is telling it to put that "A" function to apply only in given argument

★ To add new line of data in nested document

-- db.students.updateOne({ name: "Ravi", \$push: { experience: { company: "Meta", duration: 23333 } } })



Page No.	
Date	

but if you rerun the code it will be added again.

★ To add new line but if it exist it will not be added.
-- db.students.updateOne({name: "Ram", \$addToSet: {experience: {company: "Meta", duration: 2}}})

★ To remove the line from the nested document.
-- db.students.updateOne({name: "Ram", \$pull: {experience: {company: "Meta", duration: 2}}})

★ To remove the last line from the nested document
-- db.students.updateOne({name: "Ram", \$pop: {experience: 1}})

① : 1 = last line

② : -1 = first line

③ {experience: {company: "Meta", duration: 2}} ⇒ You can tell about it also

Indexing

★ Collscan ⇒ collection scan or coll scan is a scan which is used to find something from the collection in it, it scans all collection for looking what you want.

★ Ixscan ⇒ Indexing scan or Ix scan is a scan which is done when we have given indexing to collection. like it will be given to names and name will have a pointer which will point to collection. so it is faster than collscan.

★ Indexes are stored in a B-tree data structure it stores

① Index keys

② Pointers to the documents in the collection.

★ When
quick
search

★ Btree

① Store

② Index

★ The

① Sing

② Com

③ Tex

★ To

-- db.

★ To

-- db.

★ To

-- db.

Age

1

-1

★

-- db

again

be added
dataset: {experience

ment
all: {experience: 1

document
p: {experience: 1

233 => You can

scan which is
collection in
ent.

which is done
on like it
tree which

t stores

* When a query is executed, MongoDB can use the index to quickly locate the documents that match the query by searching through the B-tree

* Problems

- ① Storage space => store in two place
- ② Write performance => slow in work because giving keys, pointers and saving at same time

* There are several types of indexes available in MongoDB

- ① Single field indexes
- ② Compound indexes
- ③ Text indexes

* To see what have your query has done will executed.
-- db.teachers.find({age: { \$lte: 30 }}).explain()

* To see it more briefly
-- db.teachers.find({age: { \$lte: 30 }}).explain("executionStats")

* To create indexing

-- db.teachers.createIndex({age: 1})

age 1

Age -> it is setting Index on age

1 => positive Index

-1 => Negative Index

* To see indexing

-- db.teachers.getIndexes()

* To drop index
 -- db.teachers.dropIndex("age_1")

↓
 Name of index

* To much to write on Important writer but see slides no 28

* Aggregation

→ To write Aggregate query

→ A pipeline operation

It groups the data from multiple documents into a single document based on the specified expression

The aggregation process in MongoDB consists of several stages, each stage transforming the data in some way.

The output of one stage is fed as the input to the next stage and so on, until the final stage produces the desired result.

MongoDB provides several built-in aggregation pipeline stages to perform various operations on the data, such as \$group, \$sum, \$avg, \$min, \$max, etc.

-- db.collection.aggregate(pipeline, options)

* something same as showMany

-- db.teachers.aggregate([{\$match: {gender: 'male'}}])

↓
 tell function its it
 to function

↓
 to see result
 same as these

Group By in Mongo
 ★ -- db.teachers.aggregate([{\$group: {\$id: "\$age"}}])
 The \$group operator groups documents by the age field, creating a new document for each unique age value.

The id field in the group stage specifies the field based on which the documents will be grouped.

★ -- db.teachers.aggregate([{\$group: {\$id: "\$age", names: {\$push: "\$name"}}}])
 The names field uses the \$push operator to add the name field from each document in the group to an array.

★ -- db.teachers.aggregate([{\$group: {\$id: "\$age", students: {\$push: "\$\$ROOT"}}}])
 The \$\$ROOT value is a Reference to the current document being processed in the pipeline, which represents the complete document.

★ db.teachers.aggregate([{\$match: {\$gender: "male"}}, {\$group: {\$id: "\$age", number: {\$sum: 1}}}])
 The value of \$sum is 1, which means that for each document in the group, the value of "number" will be incremented by 1.

★ db.teachers.aggregate([{\$match: {\$gender: "male"}}, {\$group: {\$id: "\$age", countTeacher: {\$sum: 1}}, {\$sort: {\$countTeacher: -1}}])

This is used to sort the group the data.

* If you got this problem like you have to put Hobbies in the list not in nested array.
 -- aggregate ({ \$unwind: '\$Hobbies' })

* -- db.teachers.aggregate ({
 \$group: { _id: null, averageAge: { \$avg: '\$age' } } })

If you specify id: null in the \$group operator, it means that all the documents in the collection will be grouped together into a single group.

* -- db.students.aggregate ({
 \$group: { _id: null, count: { \$sum: { \$size: { \$ifNull: '\$Hobbies', [] } } } } })

If you specify \$count: { \$sum: { \$size: { \$ifNull: '\$Hobbies', [] } } } }) so ifNull is tell that if not find Hobbies leave it [] blank

* -- \$filter: {
 ① → input: <array expression>,
 ② → as: <identifier>,
 ③ → cond: <expression>

① Input: Specifies the array expression to filter.

② as: Specifies a variable name that can be used inside the cond expression to reference the current element of the input array.

③ cond: Specifies the condition that must be met in order for an element to be included in the result set. The expression

must be

* \$bucket ops
 When you want specified bin
 -- db.teachers.
 { \$bucket
 default
 } }

* \$lookup
 The \$lookup perform a left

If I have to one collection and there is
 -- db.cust - o

cust	
id	name
101	John
102	Emily
104	June

must return either true or false

lebbies

★ \$bucket operator

When you want to categorize into discrete group based on specified boundaries

-- db.teachers.aggregate([{\$match: {gender: "male"}}, {\$bucket: {groupBy: "\$age", boundaries: [0, 30, 40], default: "greater than 40", output: {count: {\$sum: 1}}}}])

means together

★ \$lookup

The \$lookup is an aggregation pipeline stage that allows you to perform a left outer join between two collections

\$ \$Null

If I have two collection cust which means customer and one collection orders which has orders so I join customer and these orders

-- db.cust.aggregate([{\$lookup: {from: "orders", localField: "id", foreignField: "customerid", as: "orderDetail"}}])

cust			orders		
id	name	email	id	Order number	customerid
101	John	John@.com	1	QR4506	101
102	Emily	Emily@.com	2	QR5046	102
104	Jane	Jane@.com	3	QR6540	103

the cond re input

den for expression

id	Name	Email	orderDetail
101	John	John@.com	
102			
104			

Null
Because no Match in orders

Not a Match

★ \$ project stage is used in the aggregation pipeline to reshape documents, include or exclude fields, and create computed fields.

It allows you to customize the output of your aggregation query by specifying which fields to include or exclude, applying expression to existing fields, and renaming fields.

-- db.emp.aggregate([{\$project: {\$id: 0, firstName: 1, monthlySalary: '\$salary', annualSalary: {\$multiply: [12, '\$salary']} } }])

- ① id: 0 ⇒ id will not be given.
- ② firstName: 1 ⇒ firstName will be given.
- ③ monthlySalary: '\$salary' ⇒ we have salary in collection, we are renaming it by MonthlySalary.
- ④ annualSalary: {\$multiply: [12, '\$salary']} ⇒ we are using multiply function that 12 * salary, salary is present in collection and name the result annualSalary.

★ .pretty()

It is used in lot of code writer as .pretty() to write the result pretty format.

★ Lapped collection

It is used to have a limit for a collection.

-- db.createCollection("order_logs", {\$lapped: true, max: 4, size: 100000})

- ① order_logs ⇒ Name of New collection.
- ② Max: 4 ⇒ max 4 document.

③ size: 100000

If we add

★ Authen

-- C:\pro

-- Secur

auth

-- Restro

-- Window

Find th

New Ma

an admin

① for th

-- db.ge

it will

② To cre

-- db.re

③ Then

-- db.

④ To

-- db.

③ size: 100000 ⇒ Maximum size 100000 bits of space (if ~~more~~)

If we add more than 4 document it will remove the first one.

★ Authentication

-- C:\Program Files\MongoDB\Server<version>\bin\mongo -fg

-- Security:

authorization: enabled

-- Restart the MongoDB Service

-- Window + R > services.msc

Find the MongoDB service Restart it

New MongoDB will not give authorized us to use it as now as an admin we have to create user that can use it

① ~~but to you database~~ -- use admin

-- db.getUsers()

it will show the users for the mongoDB

② To create user

-- ~~db.createUser({~~
~~user: "admin",~~
~~roles: [~~

-- db.createUser({
 user: "admin",
 pwd: "password",
 roles: ["useAdminDatabase",
 "dbAdminDatabase"]
 })

This will give all power

③ Then again -- use admin

-- db.getUsers()

④ To login to database

-- db.auth("admin", "password")

username

password

⑤ To logout from database
 -- db.logout()

⑥ Types of roles

① Read *

② dbAdmin

③ dbOwner

④ readWriteAnyDatabase

⑤ dbAdminAnyDatabase *

⑥ clusterMonitor

⑦ clusterAdmin

⑧ UserAdmin

⑨ root

⑩ restore

⑪ readWrite *

⑫ UserAdmin

⑬ readAnyDatabase

⑭ userAdminAnyDatabase *

⑮ clusterMonitor

⑯ hostManager

⑰ dbOwner

⑱ UserAdminAnyDatabase

⑲ backup

⑦ To drop the user
 -- db.dropuser("Rohit")

↓
 userName

⑧ without using admin in data base

-- mongo -u ram -p ram

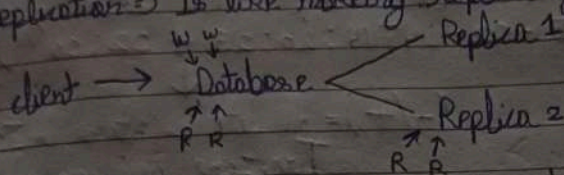
↓
 -u means username
 -p means password

⑨ ~~how to make role for~~ Make two role in one code

-- db.createUser({user: "ram", pwd: "ram", roles: [{role: "read", db: "college"}, {role: "readWrite", db: "school"}]})

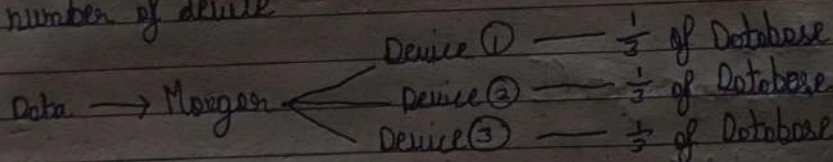
* Replication and sharding

① Replication $\Rightarrow$ Is like making replica of database.



We make Replica of Database to get answer when we get problem in main Database it will get answer (Replica) by Replica of Database get so many many calls It can give some call to Replica.

② Sharding $\Rightarrow$ In Sharding It use mangab to connect device in one database and number of Database you will connect to it will divide data in between. Device number of device.



Work fast more space.

* To insert Date in a query

-- db.students.insertOne({id: 0, name: "Ram", dob: ISODate("2000-01-01")})

↓
ISODate = To add Date.

* To insert Date and time in a query

-- db.students.insertOne({id: 1, name: "Shyam", dob: ISODate("2000-01-25T14:20:23Z")})

↓
T = time.

★ To add time ~~date~~ Date if time zone is different
 -- db.students.insertOne({id: 2, name: "Akshat", dob: ISODate("2005-02-14T14:20:23 + 02:00")})

↓
 (-/+)= all date is store in ISODate
 Timezone you can add or
 subtract the difference of time

★ To find date
 -- db.students.find({dob: {\$gte: ISODate("2000-12-12")}})

★ To group by years
 -- db.students.aggregate({\$group: {id: \$id, \$year: \$dob}})

★ To see things
 -- db.collection.aggregate({\$project: {dayofMonth: \$dayofMonth, \$dateField: \$dateField}})

★ To add
 -- db.stud

★ To form
 -- db.stude
 \$dateTo
 ????

- ① % d
- ② % m
- ③ % Y
- ④ % H
- ⑤ % M
- ⑥ % S

- ① \$ dayofMonth = dayofMonth
- ② \$ month = month
- ③ \$ dayofYear = dayofYear
- ④ \$ hour = hour
- ⑤ \$ minute = minute
- ⑥ \$ second = second
- ⑦ \$ millisecond = millisecond

★ To add current Date and time

```
-- db.students.insertOne({id: 12, name: "Amit", dob: new Date()})
```

new Date() = current Date and time

★ To format the data

```
-- db.students.aggregate([{$project: {name: 1, id: 0, dob: {$dateToString: {format: "%d/%m/%Y %H:%M:%S", date: "$dob"}}

```

\$dateToString: {format} ⇒ To set the format or format you want to

date: "\$dob" = take date from dob

- ① %d = day
- ② %m = month
- ③ %Y = Year
- ④ %H = hour
- ⑤ %M = minute
- ⑥ %S = second